CARA: Constrained Artificial Reasoning Architecture

A Protocol for Governed Epistemic Transitions in AI Systems

As these systems scale, failures are often framed as issues of hallucination, alignment, or insufficient prompting discipline.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Large language models are increasingly embedded in systems that make decisions, generate knowledge, modify code, and act across long horizons. As these systems scale, failures are often framed as issues of hallucination, alignment, or insufficient prompting discipline. This paper argues that a more fundamental class of failure has been overlooked: **unauthorized epistemic state transitions**, where claims, decisions, or artifacts acquire the *appearance* of legitimacy without passing through any explicit authority-granting process.

We introduce **CARA (Constrained Artificial Reasoning Architecture)**, a protocol-level approach to governance that treats correctness, authority, and trust as **properties that must be explicitly granted and inspected**, rather than inferred from behavior or repetition. CARA is built on a small set of primitives—**trace-first execution, phase discipline, explicit admission events, and escalation routing**—that make epistemic transitions observable and auditable across diverse workflows.

Rather than attempting to make models behave better, CARA **prevents authority from appearing silently within the governed surface**. The result is not a guarantee of correctness, but a system in which failures are detectable, responsibility is localized, and trust can be reasoned about without appeal to intuition or structural familiarity. We present the motivating failure modes, define the core primitives, and outline an illustrative reference architecture intended to apply uniformly across code, knowledge, conversational, and research systems.

---

1. Introduction: The Governance Gap

As LLM-based systems move from isolated interactions to persistent, stateful deployments, a new class of failure has become increasingly visible—but remains poorly named. These failures do not arise from obviously false outputs, malicious intent, or explicit policy violations. Instead, they emerge when **claims or decisions quietly change status** simply by being repeated, structured, or embedded within trusted-looking containers.

A speculative statement becomes a "working assumption." A draft artifact becomes "part of the knowledge base." A model suggestion becomes "the system's decision."

These transitions often occur **under good-faith use by competent operators**, not through negligence or misuse. At no point does any explicit error occur. Yet the system's epistemic state has shifted.

Most current approaches attempt to address this through better prompting, improved reasoning strategies, or post-hoc filtering. While these techniques can reduce surface-level errors, they do not address the underlying issue: **the system lacks a protocol for deciding when something is allowed to count**.

This paper refers to this failure mode as **authority laundering**: the process by which outputs acquire perceived legitimacy without an explicit, auditable authority-granting event. Authority laundering is not lying. It is not hallucination. It is a structural failure caused by missing governance semantics.

---

2. Why Behavior Is Not Enough

A growing body of work demonstrates that forcing models to externalize structure—such as defining entities or constraints before reasoning—can significantly improve reasoning performance. These approaches correctly identify that **mixing representation and reasoning increases error**. However, their primary objective remains improving *reasoning quality*, and their enforcement remains **behavioral**.

CARA addresses a different problem.

The core failure examined here is not that models reason incorrectly, but that **systems cannot determine when reasoning has exceeded its mandate**. When separation is enforced only by instruction:

- violations leave no trace,
- enforcement cannot be audited,
- and authority remains implicit.

In practice, this means that even well-structured outputs can be misused, overtrusted, or silently promoted. CARA begins from a different premise: governance

is not a property of outputs, but of **epistemic transitions**. If a claim becomes authoritative, there must be an event that made it so. If a decision is taken, the conditions under which it was taken must be visible. If a process fails or refuses, that refusal must itself be recorded.

---

3. Trace-First as a Primitive

The foundational primitive in CARA is the **trace**.

A trace is a verifiable record that *something happened*. It does not assert that the event was correct, justified, or authorized. It records only:

- what action occurred,
- under what declared mode,
- with what inputs,
- and what immediate outputs were produced.

By making traces first-class and mandatory, CARA enforces a simple invariant:

> **Within the governed surface, no epistemic state change may occur without leaving evidence.**

This invariant prevents silent failure, silent promotion, and silent refusal. It allows validation, admission, and escalation to be treated as explicit, inspectable processes rather than implicit side effects of execution.

---

4. From Traces to Governance

CARA builds governance by layering additional primitives on top of trace-first execution:

- **Phase Discipline**: separating observation, specification, witness, regime, and transformation into distinct phases that cannot be mixed without record.
- **Admission Events**: explicit transitions that grant authority to artifacts, claims, or decisions.
- **Validation and Escalation**: structured responses to violations that route failures rather than bury them.
- **Routed Context**: a hard separation between system-owned governance state and user-provided input.

These primitives do not assume a particular model, task, or domain. They define a **protocol surface** that any engine—human or machine—must pass through in order to affect governed state.

---

5. Scope and Contributions

This paper does not present a full implementation, benchmark suite, or performance evaluation. Its purpose is narrower and more foundational:

1. To name a class of failure—authority laundering—that is not addressed by existing approaches.
2. To define a minimal set of primitives that make such failures detectable and preventable within governed systems.
3. To outline an illustrative reference architecture intended to apply uniformly across code, knowledge, conversation, and research workflows.
4. To provide diagnostic canaries that distinguish protocol-level governance from behavioral discipline.

The claim is not that CARA guarantees correctness. The claim is that CARA makes **epistemic state legible**, so correctness, trust, and responsibility can be reasoned about explicitly rather than inferred.

---

6. Diagnostic Canaries: Detecting Ungoverned Systems

CARA distinguishes governed systems from ungoverned ones not by outcomes, but by **what failures are possible without leaving evidence**. To make this distinction operational, we introduce a set of **diagnostic canaries**—minimal scenarios that reveal whether a system enforces epistemic transitions or merely encourages good behavior.

These canaries are not exploits or adversarial attacks. They are **structural probes**. A system that fails them is not "unsafe" in a moral sense; it is **ungoverned** in a technical one.

6.1 Fail-Open by Absence

Scenario A reasoning or validation step expects supporting evidence, authority metadata, or prior admissions. That information is missing, malformed, or empty.

Ungoverned behavior The system proceeds anyway:

- reasoning continues,
- output is produced,
- downstream components treat the result as legitimate.

This often occurs via implicit defaults such as:

- "no evidence provided" -> skip validation
- "no objections raised" -> accept

Why this matters Absence becomes a bypass. Silence acquires power.

Canary Remove required supporting context entirely. If the system produces a result without recording a failure, refusal, or escalation, it is fail-open.

6.2 Silent Authority Promotion

Scenario An artifact is generated in a provisional or exploratory context:

- a draft answer,
- a speculative hypothesis,
- a suggested code change.

Later, the same artifact is reused, referenced, or embedded elsewhere.

Ungoverned behavior The artifact's status changes implicitly:

- "draft" becomes "accepted"
- "suggestion" becomes "decision"
- "model output" becomes "system knowledge"

No explicit event marks this transition.

Why this matters Legitimacy accumulates through reuse rather than authorization.

Canary Track an artifact across contexts. If its authority increases without a recorded admission event, authority laundering has occurred.

6.3 Exit Without Trace

Scenario A system encounters a condition it cannot or should not handle:

- unsupported mode,
- prohibited request,
- internal inconsistency.

Ungoverned behavior The system:

- returns early,
- produces a generic refusal,
- or silently drops the operation

without recording *why* execution stopped.

Why this matters Refusal itself is an epistemic event. Untraced exits erase accountability.

Canary Force a refusal path. If no trace records the refusal, the system permits invisible failure.

6.4 Context Poisoning

Scenario User-provided input includes fields that resemble governance state:

- claimed authority levels,
- fake evidence references,
- mode switches,
- trace identifiers.

Ungoverned behavior The system merges user input directly into its internal context. Downstream logic cannot distinguish verified state from untrusted claims.

Why this matters The system has no epistemic boundary. Users can shape the rules by which they are judged.

Canary Inject governance-shaped keys into user input. If they influence execution or validation without being stripped and logged, context is poisonable.

6.5 Self-Granted Legitimacy

Scenario A reasoning engine evaluates its own output:

- declares it correct,
- asserts sufficient authority,
- or marks it "validated."

Ungoverned behavior The same component:

- generates the claim,
- evaluates the claim,
- and grants it legitimacy.

No independent gate exists.

Why this matters Self-review is not review. Authority collapses into assertion.

Canary Check whether the system can approve its own outputs without an external admission step. If yes, legitimacy is self-granted.

6.6 Behavioral Compliance Masking Structural Failure

Scenario A system performs well under normal use:

- prompts are respected,
- outputs look careful,
- explanations are coherent.

Ungoverned behavior When stressed, the system:

- silently skips steps,
- invents structure,
- or promotes outputs because "nothing objected."

Why this matters Good apparent behavior hides missing protocol enforcement. The system works—until it doesn't—and no one can tell when the transition occurred.

Canary Stress the system with malformed, partial, or adversarial inputs. If failures appear only in outcomes, not traces, governance is absent.

6.7 Interpreting Canary Results

These canaries are intentionally simple. They do not require adversarial sophistication or domain expertise. Their purpose is binary:

- **If a canary triggers and leaves evidence**, the system is governable.
- **If a canary triggers silently**, the system is not.

CARA does not claim to eliminate failure. It claims to eliminate **failure without record**.

If a system's designers cannot point to where each of these failures would be recorded, the system is not governed.

---

7. Architecture Overview: A Governed Protocol Surface

CARA is not a model, a framework, or a prompting strategy. It is a **protocol surface** that constrains how reasoning systems may affect epistemic state. This section outlines the architectural shape of that surface without specifying implementation details, algorithms, or data schemas.

The goal is not to prescribe a particular system design, but to show that the primitives introduced earlier **compose coherently** into an enforceable architecture.

7.1 Separation of Roles and Responsibilities

At the highest level, CARA distinguishes between three functional roles:

- **Engines**: components that produce outputs (e.g., language models, rule-based systems, human input).
- **Governance Layer**: components that enforce phase discipline, validation, and admission.

- **State Substrate**: the persistent store of artifacts whose epistemic status matters.

Engines are **unauthorized to alter governed state by default**. They may generate content, but they cannot directly grant epistemic legitimacy. All transitions that affect epistemic status must pass through the governance layer.

This separation ensures that no engine—regardless of sophistication—can grant authority to its own outputs.

7.2 Routed Context as a Boundary

CARA enforces a hard boundary between **system-owned governance state** and **untrusted input**.

- Governance state includes:
  - declared execution mode,
  - trace identifiers,
  - authority references,
  - prior admissions.
- Untrusted input includes:
  - user prompts,
  - external data,
  - exploratory or provisional content.

The routing mechanism ensures that untrusted input **cannot write into governance state**, even if it resembles or attempts to mimic it. Any attempt to do so is stripped and recorded **in the trace**.

This boundary prevents context poisoning and ensures that all governance decisions are based on verified, system-generated information.

7.3 Phase Discipline as Structural Constraint

All governed operations occur within explicitly declared **phases**, such as observation, specification, transformation, and admission. Phases are not conceptual labels; they are **structural constraints**.

Each phase:

- has permitted inputs,
- produces bounded outputs,
- and restricts which transitions are allowed next.

Mixing phases—such as treating a speculative output as an admitted artifact—is not merely discouraged; it is **structurally invalid** within the governed surface.

Phase transitions are themselves traceable events.

7.4 Traces as the Spine of Execution

Every interaction with the governed surface produces a **trace**.

Traces are:

- append-only,
- not retroactively alterable within the governed surface,
- and generated regardless of success, failure, or refusal.

A trace records:

- the phase,
- the triggering inputs,
- the engine involved,
- and the immediate outcome.

Crucially, traces are produced **before** any validation or admission occurs. This guarantees that even aborted or rejected operations leave evidence.

Traces form the backbone of auditability, debugging, and post hoc analysis.

7.5 Validation, Admission, and Escalation

CARA distinguishes sharply between three concepts often conflated:

- **Validation**: checking whether an output satisfies declared constraints.
- **Admission**: explicitly granting epistemic authority to an artifact.
- **Escalation**: routing violations or ambiguities to a higher authority or different process.

Validation may fail without admission. Admission may be refused despite successful validation. Escalation may occur even when no explicit error exists.

Each of these is an explicit event, recorded in the trace, and governed by declared rules rather than engine discretion.

7.6 Fail-Closed Transitions

CARA is designed to fail **closed**, not open.

If required information is missing, malformed, or inconsistent:

- execution does not proceed to admission,
- refusal or escalation is recorded,

- and no epistemic state change occurs.

Silence is not treated as consent. Absence is not treated as approval.

This design directly addresses the fail-open canaries introduced earlier.

7.7 Composability Across Domains

Because CARA constrains transitions rather than outputs, the same protocol surface can be applied across domains:

- code generation systems,
- knowledge base construction,
- conversational agents,
- research and exploratory workflows.

Domain-specific logic lives outside the governance layer. CARA governs *how* conclusions are admitted, not *what* conclusions should be reached.

7.8 What This Architecture Does Not Do

For clarity, CARA does not:

- guarantee correctness,
- evaluate truth,
- replace domain expertise,
- or eliminate human judgment.

It provides the conditions under which those judgments can be made **explicit, auditable, and reversible**.

7.9 Summary

The CARA architecture establishes a simple but strict invariant:

Engines may propose. Only the protocol may decide.

By separating proposal from admission, enforcing phase discipline, and making traces unavoidable, CARA transforms reasoning systems from opaque generators into **governed participants in epistemic processes**.

---

8. Related Work and Positioning

Recent work has explored separating problem modeling from reasoning in order to improve internal reasoning quality and constraint adherence in large language

models. These approaches demonstrate that explicit structure can reduce certain classes of reasoning error by discouraging the interleaving of representation and inference.

CARA addresses a distinct and orthogonal problem. Rather than improving how models reason, CARA governs **when the outputs of any reasoning process—model-based or otherwise—are permitted to acquire epistemic authority**. The success metric is not reasoning correctness, but the absence of silent epistemic state transitions. A system employing structured or model-first reasoning may still silently promote its conclusions to authoritative status; CARA prevents such promotion structurally by requiring explicit admission events and traceable transitions.

CARA is therefore complementary to advances in reasoning quality, but operates at a different layer: the governance boundary between proposal and admission.

---

9. Conclusion

As AI systems increasingly participate in decision-making, knowledge production, and long-horizon workflows, the cost of silent epistemic failure grows. Improving reasoning quality alone does not address the question of **when reasoning is allowed to count**.

CARA reframes governance as a protocol problem. By enforcing trace-first execution, phase discipline, explicit admission, and fail-closed transitions, CARA makes epistemic state legible and contestable. The result is not correctness by fiat, but systems in which authority is granted deliberately, failures are visible, and trust can be reasoned about rather than assumed.

Corpus-Wide Reference Positioning

This paper argues that large language models are increasingly embedded in systems that make decisions, generate knowledge, modify code, and act across long horizons. Its nearest external comparison fields are ai, provenance, formal. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies,

terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Retrieval, tool use, and policy layers can reduce unsupported output without eliminating model error, source error, omission, bias, or the environmental and institutional costs of scale. A complete provenance graph can faithfully record a false or unauthorized process; provenance is evidence about lineage, not automatic truth or legitimacy. Formal assurance remains relative to the encoded model, trusted base, and refinement chain; proof does not remove specification error or unmodeled environmental behavior. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *NeurIPS 2020.* arXiv:2005.11401. https://arxiv.org/abs/2005.11401 **Relationship:** Predecessor retrieval architecture; comparison centers on provenance, admission, and fail-closed constraints not supplied by retrieval alone.

[**E2**] Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models." *ICLR 2023.* arXiv:2210.03629. https://arxiv.org/abs/2210.03629 **Relationship:** Predecessor agent pattern for interleaving reasoning and action; used to distinguish action traces from governed effects.

[**E3**] Elham Tabassi. *Artificial Intelligence Risk Management Framework (AI RMF 1.0).* NIST AI 100-1, 2023. DOI: 10.6028/NIST.AI.100-1. https://doi.org/10.6028/NIST.AI.100-1 **Relationship:** Adjacent risk-management framework used to test whether the paper operationalizes governance beyond voluntary process guidance.

[**E4**] OECD. *Recommendation of the Council on Artificial Intelligence.* OECD/LEGAL/0449, adopted 2019 and updated 2024. legalinstru-

ments.oecd.org source **Relationship:** Adjacent policy standard for accountability, traceability, transparency, safety, and human-centered values.

[**E5**] Emily M. Bender et al. "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?" *FAccT '21*, 2021. DOI: 10.1145/3442188.3445922. https://doi.org/10.1145/3442188.3445922 **Relationship:** Counterpressure concerning scale, environmental cost, documentation, and social risk; included to prevent governance proposals from becoming capability-only narratives.

[**E6**] W3C. *PROV-O: The PROV Ontology*. W3C Recommendation, 30 April 2013. https://www.w3.org/TR/prov-o/ **Relationship:** Adjacent provenance standard; it models entities, activities, and agents, but does not by itself establish the paper's stronger authority or standing claims.

[**E7**] Santiago Torres-Arias et al. "in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes." *28th USENIX Security Symposium*, 2019, pp. 1393-1410. usenix.org source **Relationship:** Predecessor supply-chain lineage system used for comparison; no influence claim is made unless the source head already documents one.

Ableman corpus

[**A1**] Adam Ableman. *A Constitution for the Context Window*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-CF26BA0E123F472F`;
source SHA-256
`60395402368E146F115CCE53D0E5A15B8FB23C90ED5B24CA92AB8515B2009965`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Governed Context & Memory (GCM)*. Ableman Corpus current edition; work identity `PAPER.PUB.18633797`;
source SHA-256
`3F17ACD8AB9C6DC66C23D22098628C70042854B198C3ABFCBFA3EC7DC27B9C08`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Retrieval-Augmented Reasoning under Constitutional Constraints*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-384489D4604BEC4D`;
source SHA-256
`133DA35C0F27F35BAA1F53DFEE415068AD2D24FF713F8CDAFA2FDC8F748AC15C`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Split-Authority AI Coordination*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-35D1BAECD5CB17AD`;
source SHA-256

06FB6133170879FA8308684F0DA33F5813EAF3B5488291C2D7D234BEB6FDFEC2.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Reasoning by Way of Receipts.* Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-04E427F6F4D94454;
source SHA-256
AF86A66CCD6A103DD75208500AE99148911A6453B6DFDF36A954279F21AB8489.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *The Epistemic Lineage Engine.* Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-E0EC3DC7EABF5060;
source SHA-256
CDD74D9AFAB6532645E68F7F6C9F0F7A9E472D9B8E717290522566FF8DC01F11.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *The Three Hash Rule.* Ableman Corpus current edition; work identity PAPER.PUB.20788617;
source SHA-256
F6FA73D083B70711A8C817F7779751D1506FEBA0E356DB9805BB1570663731B5.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Editorial source notes retained from predecessor

1. *Model-First Reasoning: Separating Modeling from Reasoning to Improve Planning and Constraint Adherence in Large Language Models.* arXiv:2512.14474.
2. Bai, Y. et al. *Constitutional AI: Harmlessness from AI Feedback.* arXiv:2212.08073.
3. Muniswamy-Reddy, K.-K. et al. *Provenance-Aware Storage Systems.* USENIX ATC, 2006.
4. Cheney, J. et al. *Provenance: A Future History.* ICDE, 2009.

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.18099715